

The Answer Is Blowing in the Wind: Using a GNN Model to Forecast Wind Turbines from the Features of Their Neighbours

Group 16

Gustas Gudaciauskas Julius Gvozdiavas Julius van Timmeren
G.Gudaciauskas-1@student.tudelft.nl J.Gvozdiavas@student.tudelft.nl J.T.Timmeren@student.tudelft.nl
Edward Olivier Francisco Ruas Vaz
E.A.Olivier@student.tudelft.nl F.RuasVaz@student.tudelft.nl

Context. Wind farm operators need power forecasts for every turbine, but SCADA data often have gaps caused by sensor failures, software issues or maintenance [1]. Simple repairs, such as backward filling [2] or interpolation, fill these gaps with guesses. Yet turbines are coupled: wake interference makes their outputs strongly correlated, and which turbines interact changes with the wind direction [2]. The turbines upwind of a silent turbine therefore still carry information about the wind about to reach it. We ask whether a model that knows which turbines are upwind can keep forecasting a turbine whose own data are missing, without first repairing the data.

Graph ML formulation. Nodes are turbines, with recent measurements (wind speed and direction, pitch, temperature, power) and a missing-data flag as features. A directed edge $A \rightarrow B$ exists when B lies downwind of A (a *wind-following graph*, which changes as the wind turns), with distance and angle to the wind as edge features. The task is node regression: predict every turbine’s future power, including turbines whose inputs are missing.

What has been done? Zhang et al. [2] forecast the power of all 134 turbines of the SDWPF farm up to 6 h ahead with a GNN that learns its graph from four similarity measures (geographic distance, dynamic time warping, maximal information coefficient and cosine similarity) and down-weights unreliable turbines with a learned credibility score; gaps are filled by backward filling before training. They note that fixed graphs miss how wake patterns change with wind direction, but their graph is learned from similarity rather than wind direction, and they do not test forecasts under turbine outages. Hammer and Barber [1] used a graph attention network on a fixed, fully connected graph of a 6-turbine farm to fill in one turbine’s missing wind speed from its neighbours, improving RMSE by about 20% over a GAN-based imputer; wind direction entered only as a node feature, and the task was imputation, not forecasting. To our knowledge, no work lets the wind direction decide which neighbours a GNN uses to forecast turbines whose own data are missing.

Hypotheses. **H1:** with missing inputs, the wind-following model loses the least accuracy, a fixed-graph model more, and non-graph and trivial strategies the most; it also matches “impute with a GNN [1], then forecast” as a single model. **H2:** the benefit is largest for turbines with several upwind neighbours and near zero for front-row turbines. **H3:** losing an upwind turbine’s data harms its downwind neighbours more than the reverse, an asymmetry that direction-blind graphs cannot show. We expect the effects to be clearest at 10 min to 1 h ahead.

Data. The SDWPF dataset [3], as used by Zhang et al. [2]: 134 turbines of 1.5 MW, 245 days of 10-minute SCADA data and relative turbine coordinates. We flag missing, unknown and abnormal records with the dataset’s rules [3], encode angles as sine/cosine and split chronologically (70/10/20%, as in [2]). Because true power is usually missing where inputs are, we *simulate* outages by setting all inputs of turbines with known power to zero, as in [1]: (i) one turbine at a time, (ii) 10–50% of turbines, (iii) patterns copied from the dataset’s real gaps.

Models. We compare our model with established approaches, from simple to advanced, all given the same data and gaps: a trivial fix (fill each gap with the next recorded value, then forecast with an LSTM), models that ignore neighbours (persistence and an LSTM per turbine), a standard graph model (GCN on a fixed distance graph), graph models that also use time (STGCN and Graph WaveNet, as benchmarked in [2]), and a two-step approach that first repairs the data with a GNN [1] and then forecasts. Our model uses the wind-following graph, learns how much to trust each upwind neighbour (GATv2 attention) and is trained with randomly hidden inputs, so it learns to rely on neighbours when a turbine goes silent.

Evaluation and experiments. MAE, RMSE and R^2 on turbines with hidden inputs, relative to complete data, averaged over several seeds: (1) accuracy vs. share of missing turbines at 10 min, 1 h and 6 h (H1); (2) degradation grouped by number of upwind neighbours (H2); (3) hiding the upwind vs. downwind turbine of a pair (H3). Code and a Jupyter notebook will be in a public GitHub repository.

ML challenge: explainability. When a turbine is hidden, which neighbours does the model rely on? Using attention weights and GNNExplainer, we test whether they lie upwind and shift with the wind.

What could go wrong? Farm-wide outages leave no neighbours to help (scenario iii). Nearby turbines are strongly correlated whatever the wind (0.81–1.0 in SDWPF [2]), so a fixed graph may do as well, which is still a useful answer. Noisy wind direction could misplace edges, so we test sensitivity to the cone angle.

References

- [1] F. Hammer and S. Barber, “Data imputation for SCADA data using graph neural networks,” *Journal of Physics: Conference Series*, vol. 3025, no. 1, p. 012014, 2025.
- [2] G. Zhang, Q. Ye, X. Li, and Z. Song, “Short-term wind power forecasting via multimodal adaptive graph neural networks with credibility-modulated aggregation,” *Information*, vol. 17, no. 7, p. 699, 2026.
- [3] J. Zhou, X. Lu, Y. Xiao, J. Tang, J. Su, Y. Li, J. Liu, J. Lyu, Y. Ma, and D. Dou, “SDWPF: A dataset for spatial dynamic wind power forecasting over a large turbine array,” *Scientific Data*, vol. 11, p. 649, 2024.